

Problem S1. Hat Circle

In a circle of N people, the person opposite seat i is at seat $i + N/2$, with wraparound. Therefore, simply compare each hat value with the value halfway around the circle.

A clean implementation loops over the first $N/2$ seats and, whenever the two opposite hats are equal, adds 2 to the answer because both people in that opposite pair see a matching hat.

Complexity. The solution runs in $O(N)$ time and uses $O(N)$ memory to store the hats.

Problem S2. Heavy-Light Composition

For each word, first count the frequency of every letter. A letter is heavy if its frequency is greater than 1, and light otherwise. The word is valid exactly when the heaviness of consecutive characters alternates for the entire string.

Thus, after the frequency count, scan the string once and check that no adjacent characters have the same heavy/light type. If every adjacent pair alternates, output T; otherwise, output F.

Complexity. Each word is processed in linear time in its length, using only a fixed-size frequency table.

Problem S3. Swipe

First compress the target array B by deleting adjacent equal values. Let the result be B' . A transformation is possible if and only if B' is a subsequence of the initial array A . If it is not, some required left swipe and right swipe must overlap in a way that destroys a needed value.

When B' is a subsequence, construct the swipes with two pointers. Match the values of B' inside A , and for each matched value determine the interval of B that should be filled by swiping from that source. Left swipes are performed in increasing order of their right endpoint. Right swipes should be saved and performed in reverse order of their left endpoint, so that later operations do not overwrite values that are still needed.

Complexity. The test and construction can be done in $O(N)$ time.

Problem S4. Painting Roads

The coloured roads must form a spanning forest of the original graph: each connected component must remain connected using coloured edges, and using fewer than $N - c$ edges is impossible for c connected components.

For the full problem, choose the spanning forest produced by DFS. In an undirected graph, every non-tree edge in a DFS forest connects a vertex to one of its ancestors; there are no cross edges. Colour each tree edge according to the parity of the depth of its child, alternating red and blue along root-to-leaf paths. Then for every grey edge, the path between its endpoints in the DFS tree alternates correctly.

Complexity. DFS and colouring take $O(N + M)$ time and memory.

Problem S5. Chocolate Bar Partition

Let μ be the mean of all cells. Subtract μ from each entry, so the goal becomes maximizing the number of connected parts whose sum is zero.

Use prefix sums for the two rows. Define $dp[k][i]$ as the best number of completed zero-sum components using the first i columns, with a possibly unfinished component sticking out in row k . The transitions either extend

the current unfinished component, close a horizontal strip, or close a component involving both rows. The sum of the unfinished component can be expressed with row prefix sums, so equal-prefix and complementary-prefix cases can be found by maps.

After processing column i , if the total prefix sum of both rows is zero, we may cut between columns i and $i + 1$ and add one completed component.

Complexity. A direct DP is quadratic. Maintaining maps from prefix-sum values to best DP values gives a near-linear implementation, typically $O(N)$ expected time with hash maps.